

IDS Pipeline Scripts Documentation

00_setup_external_capture.sh

Purpose: Configures a network interface (typically USB Ethernet adapter) to receive traffic from an external device for IDS testing.

What it does:

- Creates an isolated network (192.168.100.0/24) with the IDS machine at 192.168.100.1
- Brings up the network interface and assigns static IP
- Disables reverse path filtering (allows IDS to capture all packets regardless of routing)
- Enables promiscuous mode (captures all traffic on the network segment, not just packets addressed to this machine)
- Optimizes network buffers (increases RX ring buffer to 4096 for high traffic)
- Disables hardware offloading features (GRO, LRO, GSO, TSO) to ensure IDS sees raw, individual packets instead of aggregated ones
- **Checks firewall configuration** (UFW and iptables) and warns if rules might block traffic
- **Automatically offers to add UFW rules** if firewall is active

When to use:

- Testing IDS with PCAP files replayed from a second machine
- USB Ethernet adapters (which don't support DPDK mode)
- Creating a dedicated point-to-point network for attack traffic generation

External device setup (on the second machine):

```
sudo ip addr add 192.168.100.2/24 dev eth0
sudo ip link set eth0 up
ping 192.168.100.1 # Test connectivity
```

PROF

Requirements: Root/sudo access, interface name configured in `pipeline.conf`

Automatic features:

- Detects UFW/iptables firewall and warns if traffic might be blocked
- Offers to add UFW allow rule for the capture interface (interactive prompt)

Note: Configuration is temporary (lost on reboot). Must use AF_PACKET mode for Suricata, not DPDK since USB adapters do not support DPDK.

Connection Workflow (Step-by-Step)

Network Topology:

```
IDS Machine (192.168.100.1) <--Ethernet Cable--> External Device
(192.168.100.2)
    [USB Adapter]
    [eth0/enp2s0/etc]
```

Setup Sequence:

1. On IDS Machine - Run setup script FIRST:

```
cd /path/to/IDS/dpdk_suricata_ml_pipeline/scripts
sudo ./00_setup_external_capture.sh
# This configures the USB adapter interface with 192.168.100.1/24
```

2. Physical Connection:

- Connect Ethernet cable from IDS machine's USB adapter to external device's network port
- Cable creates Layer 2 (Ethernet) link between both devices

3. On External Device - Configure network interface:

```
# Method 1: Temporary configuration
sudo ip addr add 192.168.100.2/24 dev enp2s0 # Replace enp2s0 with
your interface
sudo ip link set enp2s0 up

# Method 2: Using ifconfig (if ip command unavailable)
sudo ifconfig enp2s0 192.168.100.2 netmask 255.255.255.0 up
```

4. Verify Connectivity (from either device):

```
# From IDS machine:
ping 192.168.100.2

# From external device:
ping 192.168.100.1

# Check ARP resolution:
arp -n | grep 192.168.100
```

5. Ready for Traffic Generation:

```
# On external device - replay PCAP to IDS:
sudo tcpreplay -i eth0 --mbps 10 attack_traffic.pcap
```

How It Works:

- **Layer 2:** Both devices are on same Ethernet segment (direct cable connection)
- **Layer 3:** Both devices on 192.168.100.0/24 subnet (can route to each other)
- **IDS Capture:** USB adapter in promiscuous mode captures ALL packets on the segment
- **Traffic Flow:** External device sends packets → Cable → USB adapter → Suricata → Kafka → ML Pipeline

Troubleshooting:

- **No ping response:** Check cable, interface up status, IP configuration, firewall rules
- **Wrong interface name:** Use `ip link show` to find correct interface name
- **Permission denied:** Ensure sudo/root access on both machines
- **Firewall blocking:** Disable firewall temporarily or add allow rules

Note: Configuration is temporary (lost on reboot). Must use AF_PACKET mode for Suricata, not DPDK since USB adapters do not support DPDK.

01_bind_interface.sh

Purpose: Binds a network interface to DPDK (Data Plane Development Kit) userspace drivers for high-performance packet processing.

What it does:

- Takes a Linux network interface OFFLINE and transfers control from kernel drivers to DPDK userspace drivers
- Automatically detects PCI address of the network interface
- Unbinds interface from current kernel driver (e.g., e1000e, igb, ixgbe)
- Loads appropriate DPDK driver module (vfio-pci, uio_pci_generic, or igb_uio)
- Binds interface to DPDK driver so Suricata can use DPDK mode for faster packet processing
- Backs up original interface configuration for restoration later
- Handles IOMMU/NOIOMMU mode configuration for vfio-pci

When to use:

- Running Suricata in **DPDK mode** for maximum performance (millions of packets/sec)
- Using network interfaces that support DPDK (Intel 1GbE/10GbE NICs, some Broadcom/Mellanox cards)
- Production environments requiring line-rate packet capture and processing

⚠ CRITICAL WARNINGS:

- **Interface goes completely OFFLINE** - no SSH, no normal networking on that interface
- **DO NOT use on your primary/management interface** or you'll lose remote access
- Use a dedicated capture interface or secondary NIC
- Cannot be used with USB Ethernet adapters (use AF_PACKET mode instead)

Supported DPDK drivers (configured in `pipeline.conf`):

1. **vfio-pci** (recommended): Modern, secure driver with IOMMU support
2. **uio_pci_generic**: Generic UIO driver, widely compatible
3. **igb_uio**: Legacy DPDK driver, requires custom kernel module

Configuration requirements:

- Interface name in `pipeline.conf` (e.g., `NETWORK_INTERFACE=ens33`)
- DPDK driver selection (e.g., `DPDK_DRIVER=vfio-pci`)
- Optional: PCI address (auto-detected if not specified)
- Backup enabled by default (`BACKUP_INTERFACE_CONFIG=true`)

How it works:

1. Validates interface exists and gets PCI address (e.g., 0000:02:00.0)
2. Checks current driver and verifies DPDK driver availability
3. Takes interface down with `ip link set down`
4. Loads kernel module for target DPDK driver
5. Uses `dpdk-devbind.py` to unbind from kernel driver
6. Uses `dpdk-devbind.py` to bind to DPDK driver
7. Verifies binding succeeded with `lspci`

Restoration:

- Run `unbind_interface.sh` to restore interface to kernel driver
- Backup saved to `logs/interface_backup/backup_TIMESTAMP.conf`
- Contains original driver name and interface configuration

Prerequisites:

- DPDK installed (`dpdk-devbind.py` available)
- Root/sudo access
- Kernel modules available (`vfio-pci/uio_pci_generic` built into kernel or as modules)
- For `igb_uio`: Custom module must be compiled and installed

After binding:

- Interface disappears from `ip link show` output
- Visible in `dpdk-devbind.py --status` under DPDK-compatible devices
- Start Suricata with `03_start_suricata.sh` (DPDK mode)
- Suricata directly controls interface via DPDK PMD (Poll Mode Driver)

Example output:

```
Interface: ens33
PCI Address: 0000:02:00.0
Current Driver: e1000e
Target Driver: vfio-pci
```

⚠ WARNING ⚠

This will bind ens33 to DPDK driver.
The interface will be taken OFFLINE and unavailable for normal use!
Continue? (type 'yes' to proceed):

Note: This is ONLY for DPDK mode. For AF_PACKET mode (USB adapters), use `00_setup_external_capture.sh` instead - no binding required.

02_setup_kafka.sh

Purpose: Installs and configures Apache Kafka message broker for streaming IDS events between Suricata and the ML pipeline.

What it does:

- **Detects or installs Kafka:** Checks if Kafka already exists at `/opt/kafka` or in PATH, downloads Kafka 3.6.0 if needed
- **Starts Kafka services:** Launches Zookeeper (dependency) and Kafka broker in daemon mode
- **Creates required topics:** Sets up two Kafka topics with 3 partitions each:
 - `suricata-alerts` - Receives raw Suricata alerts and flow events
 - `ml-predictions` - Receives ML-enhanced alerts with threat scores
- **Python library installation:** Installs `kafka-python` and `confluent-kafka` into project's virtual environment
- **Configuration management:** Reads topic names and bootstrap servers from `pipeline.conf`

When to use:

- First-time IDS setup (run once to install Kafka)
- After system reboot (to restart Kafka services)
- Creating new Kafka topics for additional data streams
- Fixing broken Kafka installation

How it works:

1. Checks if Kafka is installed (`/opt/kafka` or `kafka-server-start.sh` in PATH)
2. If not found, downloads from Apache mirror and extracts to `/opt/kafka`
3. Starts Zookeeper on port 2181 (Kafka's coordination service)
4. Starts Kafka broker on port 9092 (default message broker port)
5. Creates topics with `kafka-topics.sh --create` (uses `--if-not-exists` to avoid errors)
6. Installs Python Kafka clients in virtual environment for producer/consumer code

Kafka topics explained:

- **suricata-alerts:** Suricata writes JSON events here via EVE output plugin
 - Contains: Flow events, DNS queries, HTTP logs, TLS handshakes, alerts
 - Consumed by: `ml_kafka_consumer.py` for ML inference
- **ml-predictions:** ML consumer writes enhanced alerts here
 - Contains: Original Suricata alert + ML prediction + confidence score + threat level

- Consumed by: Downstream SIEM systems, alert dashboards, logging services

Configuration (from `pipeline.conf`):

```
KAFKA_BOOTSTRAP_SERVERS="localhost:9092"  
KAFKA_TOPIC_ALERTS="suricata-alerts"  
KAFKA_TOPIC_ML_PREDICTIONS="ml-predictions"
```

Partitions = 3: Allows parallel processing with multiple consumer threads (higher throughput)

Replication factor = 1: Single Kafka broker (no redundancy), suitable for testing/development

Prerequisites:

- Java Runtime Environment (JRE) for Kafka
- Internet connection (for downloading Kafka if not installed)
- Python 3 virtual environment (auto-created if missing)
- Ports 2181 (Zookeeper) and 9092 (Kafka) available

Stopping Kafka:

```
/opt/kafka/bin/kafka-server-stop.sh      # Stop Kafka broker  
/opt/kafka/bin/zookeeper-server-stop.sh  # Stop Zookeeper
```

Common issues:

- **Port 9092 already in use:** Kafka already running, or another service using the port
- **Connection refused:** Zookeeper not started before Kafka, or firewall blocking
- **Topic already exists warning:** Normal if re-running script, topics persist after creation

Note: Kafka runs in daemon mode (background), survives terminal closure but NOT system reboot. Add to systemd for auto-start on boot.

PROF

03_start_suricata_afpacket.sh

Purpose: Starts Suricata IDS in AF_PACKET mode - software-based packet capture compatible with ALL network interfaces including USB adapters.

What it does:

- Validates interface exists and brings it UP if needed
- Checks if Suricata is already running (offers to kill existing process)
- Starts Suricata in daemon mode (-D) with AF_PACKET capture on specified interface
- Configures EVE JSON output to `eve.json` for Kafka consumption
- Sets worker threads for parallel packet processing
- Uses cluster_flow mode for load balancing across threads

- Displays PID, log file locations, and monitoring commands

When to use:

-  **USB Ethernet adapters** (enx00e04c36074c, etc.)
-  **Virtual machines** without PCI passthrough
-  **Testing/development** environments
-  **Any network interface** that doesn't support DPDK
-  **When you need the interface to remain usable** for normal networking

How it works:

1. Validates interface exists with `ip link show`
2. Brings interface UP if down
3. Kills existing Suricata process if running (optional)
4. Launches Suricata with these key parameters:
 - `--af-packet="$NETWORK_INTERFACE"` - Uses kernel's AF_PACKET API
 - `-D` - Daemon mode (background)
 - `--set af-packet.0.threads="$SURICATA_CORES"` - Parallel processing
 - `--set af-packet.0.cluster-type=cluster_flow` - Load balance by flow (5-tuple hash)
 - `--set outputs.5.eve-log.enabled=yes` - Enable EVE JSON logging
5. Waits 3 seconds and verifies Suricata PID exists

Configuration (from `pipeline.conf`):

```
NETWORK_INTERFACE="enx00e04c36074c" # Your capture interface
SURICATA_LOG_DIR="../../logs/suricata"
SURICATA_HOME_NET="192.168.100.0/24"
SURICATA_CORES="4" # Worker threads
```

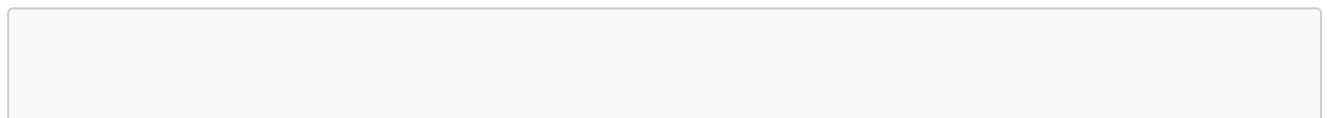
Output files:

- `eve.json` - All events in JSON format (alerts, flows, DNS, HTTP, TLS)
- `fast.log` - Quick alert summary (one line per alert)
- `stats.log` - Performance counters and statistics

Performance characteristics:

- **Throughput:** ~100-500 Mbps depending on hardware
- **CPU overhead:** Moderate (kernel context switches)
- **Latency:** Higher than DPDK (milliseconds)
- **Compatibility:** Works with ANY interface

Monitoring:



```
# Watch alerts in real-time
tail -f ../logs/suricata/eve.json | jq .

# Check Suricata is running
ps aux | grep suricata

# View statistics
suricatasc -c dump-counters
```

Note: Interface remains available for normal networking (SSH, ping, etc.) unlike DPDK mode.

03_start_suricata_dpdk.sh

Purpose: Starts Suricata IDS in DPDK mode - high-performance userspace packet capture for line-rate processing.

What it does:

- Validates Suricata was compiled with DPDK support (**--build-info**)
- Checks interface is bound to DPDK driver (uses **dpdk-devbind.py --status**)
- Optionally starts Kafka if not running
- Creates comprehensive DPDK configuration with Kafka output
- Tests configuration validity before starting
- Launches Suricata in DPDK mode with PCI address-based interface access
- Configures direct Kafka streaming (no file intermediary)

When to use:

- **✓ Production environments** requiring maximum performance
- **✓ High-speed networks** (1 Gbps+, 10 Gbps ideal)
- **✓ Dedicated capture NICs** (Intel 1GbE/10GbE, Broadcom, Mellanox)
- **✓ Line-rate packet processing** (millions of packets/sec)
- **✗ NOT for USB adapters** (incompatible with DPDK)
- **✗ NOT for interfaces you need for SSH/management**

Prerequisites (MUST run first):

1. **01_bind_interface.sh** - Bind interface to DPDK driver
2. **02_setup_kafka.sh** - Start Kafka broker
3. Suricata compiled with **--enable-dpdk** flag
4. DPDK-compatible NIC (check with **dpdk-devbind.py --status**)

How it works:

1. Validates DPDK support with **suricata --build-info | grep DPDK**
2. Checks interface binding status via **dpdk-devbind.py**
3. Creates/updates Suricata config with DPDK and Kafka sections:
 - **DPDK EAL params:** Process type, memory allocation

- **Interface config:** Uses PCI address (0000:02:00.0), not interface name
 - **Kafka output:** Direct streaming to `suricata-alerts` topic with Snappy compression
 - **Flow logging:** Captures ALL flows for ML feature extraction
4. Tests config with `suricata -T -c config.yaml --dppk`
 5. Starts Suricata: `suricata -c config.yaml --dppk -l logs/`

Configuration auto-generated:

```
dppk:
  eal-params:
    proc-type: primary
  interfaces:
    - interface: 0000:02:00.0 # PCI address, NOT eth0!
      threads: 4
      cluster-type: cluster_flow
      promisc: yes

outputs:
  - eve-log:
      filetype: kafka # Direct Kafka streaming
      kafka:
        bootstrap-servers: localhost:9092
        topic: suricata-alerts
        compression-codec: snappy
      types:
        - alert
        - flow # For ML feature extraction
        - dns
        - http
        - tls
```

Performance characteristics:

- **Throughput:** 1-10+ Gbps (hardware-limited)
- **CPU overhead:** Low (no kernel, zero-copy)
- **Latency:** Microseconds
- **Compatibility:** Intel, Broadcom, Mellanox NICs only

Verification:

```
# Check DPDK support
suricata --build-info | grep DPDK

# Check interface binding
dppk-devbind.py --status

# Monitor Suricata stats
suricatasc -c stats
```

```
# Watch Kafka output
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
suricata-alerts
```

Troubleshooting:

- **"DPDK support: no"**: Recompile Suricata with `--enable-dpdk`
- **"No interfaces bound"**: Run `01_bind_interface.sh` first
- **"EAL: Cannot open /dev/uis"**: Load correct kernel module (vfio-pci/uis_pci_generic)
- **Config test fails**: Check PCI address matches bound interface

Note: Interface is OFFLINE for normal use - no ping, SSH, or standard networking while in DPDK mode.

Comparison: AF_PACKET vs DPDK Mode

When to Use AF_PACKET (`03_start_suricata_afpacket.sh`)

Use Case	Why AF_PACKET
USB Ethernet adapters	DPDK doesn't support USB devices
Testing/Development	Easier setup, no binding required
Low-moderate traffic	< 500 Mbps sustained
Need interface for other tasks	Interface stays available for SSH/ping
Virtual machines	No PCI passthrough needed
Any network interface	Universal compatibility

When to Use DPDK (`03_start_suricata_dpdk.sh`)

Use Case	Why DPDK
Production IDS	Maximum performance and efficiency
High-speed networks	1-10+ Gbps line-rate capture
Dedicated capture NIC	Interface only for IDS, nothing else
Intel/Broadcom NICs	Hardware acceleration support
Minimal CPU overhead	Zero-copy, kernel bypass
Mission-critical monitoring	Microsecond latency, no packet drops

Technical Differences

AF_PACKET Mode:

- **Capture method:** Linux kernel's AF_PACKET socket API

- **Driver location:** Kernel space (standard network drivers)
- **Interface state:** Remains UP and usable for normal networking
- **Performance:** ~100-500 Mbps (kernel overhead, context switches)
- **Setup:** Simple - just start Suricata, no binding
- **Compatibility:** ALL network interfaces (eth0, wlan0, USB adapters)

DPDK Mode:

- **Capture method:** DPDK Poll Mode Driver (PMD) in userspace
- **Driver location:** Userspace (vfio-pci/uiopci_generic)
- **Interface state:** OFFLINE - taken over by DPDK, unusable for normal networking
- **Performance:** 1-10+ Gbps (zero-copy, kernel bypass, no interrupts)
- **Setup:** Complex - bind interface with `01_bind_interface.sh` first
- **Compatibility:** ONLY DPDK-compatible NICs (Intel 1G/10G, some Broadcom/Mellanox)

Quick Decision Tree

```
Are you using a USB Ethernet adapter?
└─ YES → Use AF_PACKET (03_start_suricata_afpacket.sh)
└─ NO → Do you need > 500 Mbps throughput?
    └─ YES → Use DPDK (03_start_suricata_dpdk.sh) + 01_bind_interface.sh
    └─ NO → Use AF_PACKET (simpler, sufficient performance)
```

Current Project Setup: Using AF_PACKET mode with USB adapter `enx00e04c36074c` on 192.168.100.0/24 network.

04 start kafka bridge.sh

Purpose: Starts a file-to-Kafka bridge that reads Suricata's EVE JSON log file and streams events to Kafka in real-time.

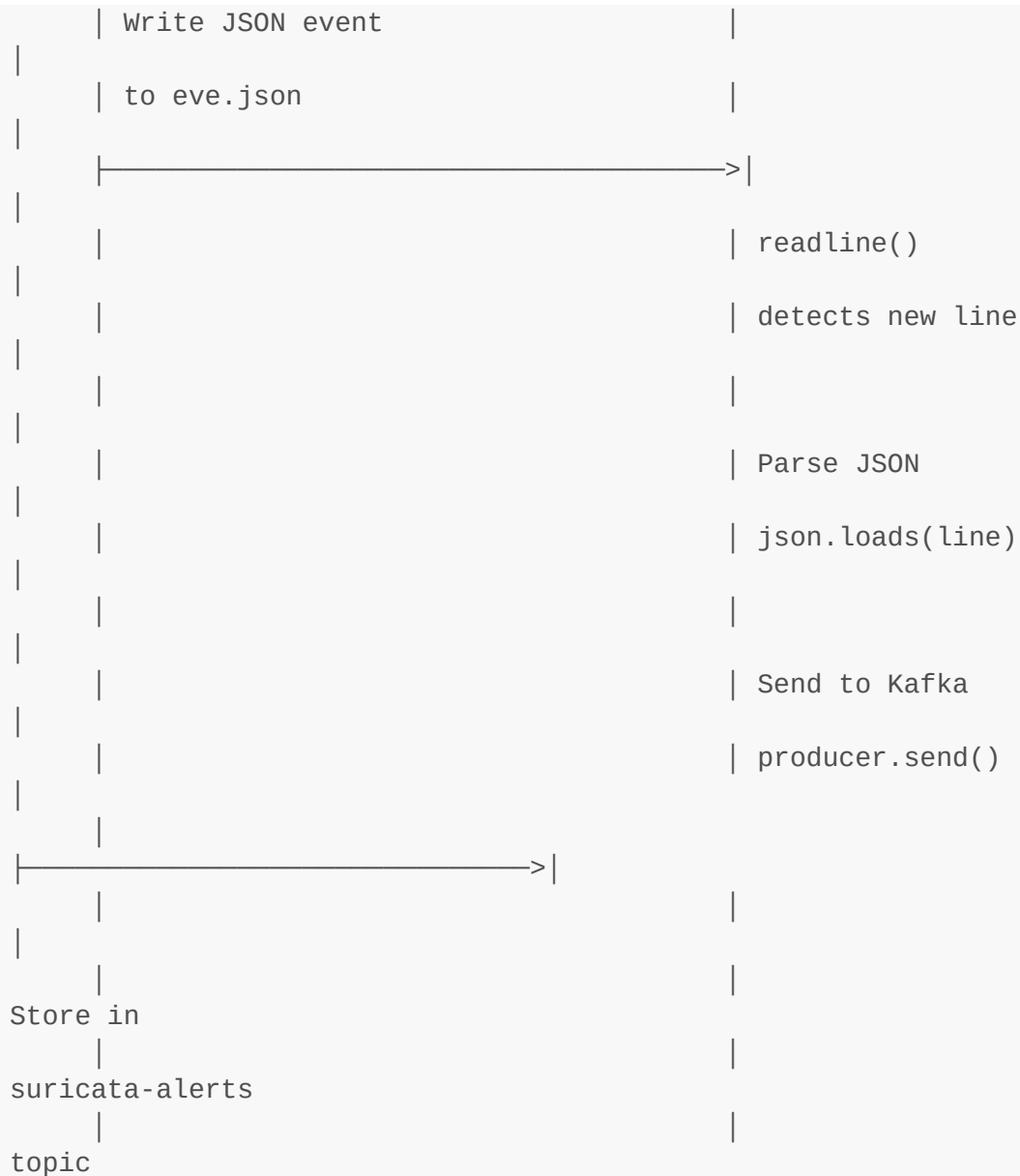
PROF

What it does:

- Validates Suricata and Kafka are both running
- Activates Python virtual environment
- Checks/installs kafka-python dependency
- Kills existing bridge process if running (optional)
- Starts `suricata_kafka_bridge.py` in background daemon mode
- Monitors file changes and publishes to Kafka topic

Data Flow





When to use:

- **Legacy/Fallback mode:** When Suricata doesn't have native Kafka output support
- When Suricata is configured to write EVE JSON to file instead of directly to Kafka
- Debugging/testing when you want to inspect logs before Kafka ingestion
- Systems where recompiling Suricata with Kafka support isn't possible

How it works:

1. Validates Suricata is running with `ps aux | grep suricata`
2. Validates Kafka is accessible on port 9092
3. Checks for existing bridge process with `pgrep -f suricata_kafka_bridge.py`
4. Activates virtual environment and ensures kafka-python is installed
5. Starts `suricata_kafka_bridge.py` which:
 - Opens Suricata's `eve.json` log file
 - Tails the file (like `tail -f`) to watch for new lines
 - Parses each JSON event

- Publishes to Kafka topic `suricata-alerts`
- Handles file rotation and continues reading

⚠ Important: When is this needed?

Modern Setup (Preferred - DPDK mode):

```
Suricata → Direct Kafka output → suricata-alerts topic  
# No bridge needed! Suricata writes directly to Kafka
```

Legacy/AF_PACKET Setup:

```
Suricata → eve.json file → Bridge script → suricata-alerts topic  
# Bridge needed because Suricata configured for file output
```

Your current setup: If you're using `03_start_suricata_afpacket.sh`, it configures Suricata to write to `eve.json` file, so **this bridge IS needed** to get events into Kafka.

If using `03_start_suricata_dpdk.sh`, it configures direct Kafka output, so **this bridge is NOT needed**.

Configuration:

The bridge script (`suricata_kafka_bridge.py`) typically monitors:

- **Input:** `/var/log/suricata/eve.json` or `../logs/suricata/eve.json`
- **Output:** Kafka topic `suricata-alerts` on `localhost:9092`

Prerequisites:

- Suricata running and generating `eve.json` logs
- Kafka running on port 9092
- Python virtual environment with `kafka-python`
- Read access to Suricata log directory

Process management:

```
# Check if running  
ps aux | grep suricata_kafka_bridge.py  
  
# Stop bridge  
pkill -f suricata_kafka_bridge.py  
  
# Monitor logs  
tail -f ../logs/bridge/bridge.log
```

Performance considerations:

- **Latency:** Adds small delay (milliseconds) compared to direct Kafka output
- **Reliability:** Depends on file I/O and disk performance
- **File rotation:** Bridge must handle Suricata log rotation gracefully
- **Overhead:** Extra process, disk writes, file reads

Advantages of using bridge:

-  Can inspect/debug events in file before Kafka
-  Works with any Suricata installation (no recompilation needed)
-  Allows multiple consumers of same log file
-  Can replay historical logs by restarting bridge

Disadvantages vs direct Kafka:

- × Higher latency (file I/O overhead)
- × Extra disk usage for log files
- × Additional process to monitor
- × Potential for data loss if bridge crashes

05_start_ml_consumer.sh

Purpose: Starts the ML inference consumer that reads Suricata events from Kafka, performs ML predictions, and publishes enhanced alerts back to Kafka.

What it does:

- Activates Python virtual environment with required ML libraries
- Validates dependencies (kafka-python, joblib, scikit-learn, etc.)
- Checks Kafka is running on port 9092
- Verifies ML model file exists (or uses default)
- Kills existing ML consumer process if running (optional)
- Starts `ml_kafka_consumer.py` in background daemon mode
- Monitors startup and displays process information

When to use:

- After starting Suricata (03_start_suricata_*.sh) to enable ML-enhanced detection
- When you want real-time ML inference on network flows
- After training new models and deploying them
- When restarting the ML pipeline after changes

How it works:

1. Sources Python virtual environment at `../venv/bin/activate`
2. Installs missing dependencies (kafka-python, confluent-kafka, joblib)
3. Validates Kafka connectivity with `netstat -tuln | grep :9092`
4. Changes to `../src` directory where `ml_kafka_consumer.py` lives
5. Launches consumer with `nohup python3 ml_kafka_consumer.py` in background
6. Passes config file path and optional model path as command-line arguments

7. Redirects output to `../logs/ml/ml_consumer.out`
8. Waits 2 seconds and verifies process is still running

Data flow:

```
Kafka Topic (suricata-alerts)
↓
ml_kafka_consumer.py reads events
↓
Extracts 65 CICIDS2017 features
↓
Maps to 34 core features
↓
Runs ML inference (Random Forest / LightGBM / Ensemble)
↓
Combines Suricata alert + ML prediction + confidence score
↓
Publishes to Kafka Topic (ml-predictions)
```

What the consumer does internally (ml_kafka_consumer.py):

1. **Consumes events:** Polls Kafka for Suricata flow events and alerts
2. **Feature extraction:** Converts flow metadata to 65 CICIDS2017 features
3. **Feature mapping:** Reduces to 34 core features for model compatibility
4. **ML inference:** Runs prediction with loaded model (Random Forest/LightGBM)
5. **Alert enhancement:** Combines Suricata severity + ML confidence + threat level
6. **Kafka publishing:** Sends enhanced alerts to `ml-predictions` topic
7. **Performance tracking:** Logs latency, throughput, confidence distribution

Prerequisites:

- Python 3.8+ virtual environment with ML libraries
- Kafka running (port 9092)
- Suricata generating events to `suricata-alerts` topic
- ML model files in `/ML Models/` directory
- Dependencies: kafka-python, joblib, scikit-learn, numpy, pandas

Performance monitoring:

```
# Watch consumer logs
tail -f ../logs/ml/ml_consumer.log

# Watch output stream
tail -f ../logs/ml/ml_consumer.out

# Monitor predictions in Kafka
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic ml-
predictions | jq .
```

Process management:

```
# Check if running
ps aux | grep ml_kafka_consumer.py

# Stop consumer
pkill -f ml_kafka_consumer.py

# Restart consumer
./04_start_ml_consumer.sh
```

Output format (ml-predictions topic):

```
{
  "timestamp": "2025-10-25T10:30:45.123Z",
  "original_alert": { /* Suricata alert data */ },
  "ml_prediction": "BENIGN" | "ATTACK",
  "attack_category": "DoS" | "Portscan" | "Brute Force" | "Web Attack",
  "confidence": 0.95,
  "threat_level": "HIGH" | "MEDIUM" | "LOW",
  "threat_score": 8.5,
  "model_used": "random_forest_2017",
  "features_extracted": 65,
  "processing_time_ms": 12.5
}
```

Common issues:

- **Virtual environment not found:** Run `python3 -m venv ../../venv` to create it
- **Kafka connection refused:** Start Kafka with `02_setup_kafka.sh`
- **Model file not found:** Check path in `pipeline.conf` or use default in `/ML Models/`
- **Import errors:** Install requirements: `pip install -r requirements.txt`
- **Already running:** Kill existing process before restarting

Note: Consumer runs continuously in background. Must be manually stopped with `pkill` or by killing the process. Logs rotate automatically to prevent disk fill.

Typical startup sequence (with bridge):

```
# 1. Setup interface
sudo ./00_setup_external_capture.sh

# 2. Start Kafka
./02_setup_kafka.sh
```

```
# 3. Start Suricata (file output mode)
sudo ./03_start_suricata_afpacket.sh

# 4. Start bridge (file → Kafka)
./04_start_kafka_bridge.sh

# 5. Start ML consumer (Kafka → ML → Kafka)
./05_start_ml_consumer.sh
```

Monitoring bridge operation:

```
# Watch bridge processing events
tail -f ../logs/bridge/bridge.log

# Check Suricata generating events
tail -f ../logs/suricata/eve.json

# Verify events reaching Kafka
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic
suricata-alerts --from-beginning
```

Common issues:

- **File not found:** Check Suricata log path in bridge script
- **Permission denied:** Bridge needs read access to eve.json
- **Kafka connection refused:** Ensure Kafka is running
- **No events:** Check Suricata is generating traffic and writing to eve.json
- **Duplicate events:** Bridge might have restarted and replaying old events

Note: If you modify Suricata to output directly to Kafka (like in DPDK mode), you can skip this bridge entirely - it's only needed when Suricata writes to file instead of Kafka.

PROF

monitor_traffic.sh

Purpose: Interactive monitoring dashboard for real-time IDS pipeline visibility - watch alerts, flows, DNS, HTTP, and ML predictions as they happen.

What it does:

- Provides 10 interactive monitoring modes via menu-driven interface
- Displays live Suricata events with color-coded, formatted JSON output
- Shows real-time statistics and analytics on network traffic
- Monitors ML consumer predictions and Kafka streams
- Searches historical data by IP address or event type
- Uses `jq` for pretty-printing JSON and extracting specific fields

When to use:

- **Debugging:** Watch events in real-time to verify IDS is detecting traffic
- **Demonstration:** Show live attack detection for presentations/demos
- **Troubleshooting:** Check if Suricata, Kafka, or ML consumer are working
- **Analysis:** View statistics on traffic patterns and alert trends
- **Testing:** Validate attack traffic generates expected alerts

Monitoring modes:

1. **Live Suricata Events (all types)** - Everything: alerts, flows, DNS, HTTP, TLS, stats
2. **Live Alerts Only** - Signature-based detections with severity, category, source/dest
3. **Live Flow Events** - Network flows with packet/byte counts and TCP states
4. **Live DNS Queries** - DNS lookups with query names, types, and resolved IPs
5. **Live HTTP Requests** - HTTP traffic with methods, URLs, hosts, status codes
6. **Event Statistics** - Counts by event type, severity, top IPs, top signatures
7. **Recent Alerts (last 20)** - Quick view of most recent detections
8. **Search by IP Address** - Filter all events involving specific IP
9. **ML Consumer Output** - Live ML predictions with confidence scores and threat levels
10. **Kafka Events Stream** - Raw Kafka topic data (suricata-alerts or ml-predictions)

How it works:

1. Presents interactive menu with colored options
2. User selects monitoring mode (1-10)
3. Script uses `tail -f`, `grep`, and `jq` to:
 - Stream log files in real-time
 - Filter by event type
 - Extract relevant JSON fields
 - Format output with colors
4. Press Ctrl+C to stop live monitoring and return to terminal

Example usage scenarios:

PROF

Scenario 1: Verify IDS is detecting attacks

```
./monitor_traffic.sh
# Select option 2 (Live Alerts Only)
# Run attack traffic from external device
# Watch alerts appear in real-time with severity and signature
```

Scenario 2: Debug why ML consumer isn't working

```
./monitor_traffic.sh
# Select option 10 (Kafka Events Stream) → suricata-alerts
# Verify Suricata is sending events to Kafka
```

```
# Then select option 9 (ML Consumer Output)
# Check for errors or missing predictions
```

Scenario 3: Analyze traffic patterns

```
./monitor_traffic.sh
# Select option 6 (Event Statistics)
# See: Top source IPs, top destinations, alert distribution
# Identify suspicious patterns or heavy talkers
```

Prerequisites:

- Suricata running and generating **eve.json** logs
- **jq** installed for JSON parsing (script uses it heavily)
- Read access to **/var/log/suricata/eve.json**
- For Kafka monitoring: Kafka running and accessible

Installation check:

```
# Check if jq is installed
which jq || sudo apt-get install jq

# Make script executable
chmod +x monitor_traffic.sh

# Run
./monitor_traffic.sh
```

Key features:

- **✓ Color-coded output:** Different colors for alerts (red), flows (green), DNS (magenta), HTTP (cyan)
- **✓ Real-time streaming:** Uses **tail -f** to continuously monitor logs
- **✓ Structured output:** **jq** extracts only relevant fields, removes noise
- **✓ No root required:** Reads log files, doesn't modify anything
- **✓ Ctrl+C friendly:** Clean exit from any monitoring mode

Common use cases:

Task	Option	Why
"Is IDS working?"	2 (Live Alerts)	See if attacks trigger alerts
"Is traffic reaching IDS?"	3 (Live Flows)	Verify network flows captured
"What domains being accessed?"	4 (Live DNS)	Monitor DNS queries

Task	Option	Why
"Check HTTP traffic"	5 (Live HTTP)	See web requests
"Overall system health"	6 (Statistics)	Event counts and top talkers
"Did specific IP attack?"	8 (Search by IP)	Filter by source/dest IP
"Is ML working?"	9 (ML Consumer)	See predictions and confidence
"Raw Kafka data"	10 (Kafka Stream)	Debug Kafka integration

Troubleshooting:

- **"jq: command not found"**: Install with `sudo apt-get install jq`
- **"No such file"**: Check Suricata log path, might be `../logs/suricata/eve.json`
- **No events showing**: Generate traffic or check if Suricata is running
- **Kafka consumer fails**: Verify Kafka is running on localhost:9092
- **Permission denied**: Need read access to Suricata logs directory

Pro tips:

```
# Monitor alerts in one terminal, flows in another
terminal1$ ./monitor_traffic.sh # Select option 2
terminal2$ ./monitor_traffic.sh # Select option 3

# Save statistics to file
./monitor_traffic.sh # Select option 6 > stats.txt

# Grep for specific signature
tail -f /var/log/suricata/eve.json | grep "SQL Injection" | jq .
```

Note: This is a **read-only monitoring tool** - it doesn't modify logs, stop processes, or change configuration. Safe to run anytime for visibility into IDS operations.

PROF

status_check.sh

Purpose: Quick health check for all IDS pipeline components - shows what's running, what's not, and system resource usage.

What it does:

- Checks DPDK installation and bound interfaces
- Verifies Kafka is running on port 9092 and lists topics
- Confirms Suricata is active with PID and recent stats
- Validates ML consumer process is running
- Displays system resources (CPU, memory, disk usage)
- Provides overall pipeline status summary (X/4 components running)

When to use:

- **Quick diagnosis:** "Is everything running?"
- **After startup:** Verify all components started successfully
- **Troubleshooting:** Identify which component is down
- **Before running traffic:** Confirm pipeline is ready
- **System monitoring:** Check resource consumption

How it works:

1. Checks each component in sequence:
 - DPDK: Runs `dpdk-devbind.py --status` to find bound devices
 - Kafka: Tests port 9092 with `netstat`, lists topics if running
 - Suricata: Searches for process with `pgrep`, shows recent stats
 - ML Consumer: Looks for `ml_kafka_consumer.py` process
2. Displays system resource usage with `top`, `free`, `df`
3. Counts running components (0-4) and provides status summary
4. Suggests quick action commands (`start_all`, `stop_all`, `view_logs`)

Output sections:

1. DPDK Status:

► DPDK Status

✓ DPDK installed and 1 device(s) bound
Network devices using DPDK-compatible driver
0000:02:00.0 'Intel 82599ES' drv=vfio-pci

HugePages_Total: 1024
HugePages_Free: 512

2. Kafka Status:

► Kafka Status

✓ Kafka running on port 9092

Topics:
suricata-alerts
ml-predictions

3. Suricata Status:

► Suricata Status

- ✓ Suricata running (PID: 12345)
- ✓ DPDK support enabled

Recent stats (last 5 lines):
[stats counters shown here]

4. ML Consumer Status:

► ML Consumer Status

- ✓ ML Consumer running (PID: 12346)

Recent activity (last 5 lines):
[consumer log output]

5. System Resources:

► System Resources

CPU Usage:
%Cpu(s): 12.5 us, 5.3 sy, 0.0 ni, 80.2 id

Memory Usage:
Mem: 15Gi 8.2Gi 2.1Gi 1.5Gi 4.7Gi

Disk Usage:
Used: 45G / 100G (45%)

6. Pipeline Summary:

► Pipeline Summary

- ✓ Full pipeline operational (4/4 components)

Status indicators:

- ✓ (Green) = Component running/OK
- ⚠ (Yellow) = Component not running or partial status
- ✖ (Red) = Component failed or missing

Use cases:

Scenario	What to Check
Just started pipeline	All 4 components should be ✓

Scenario	What to Check
No alerts appearing	Check Suricata and ML Consumer are running
High CPU usage	Look at System Resources section
Kafka errors	Verify Kafka shows ✓ with topics listed
DPDK issues	Check DPDK section shows bound devices

Prerequisites:

- None - script is read-only and safe to run anytime
- May need sudo for full DPDK/Suricata information

Quick actions provided:

```
Start all: ./scripts/start_all.sh      # Start all components
Stop all:  sudo ./scripts/stop_all.sh  # Stop everything
Logs:     ./scripts/view_logs.sh      # View log files
```

Example workflow:

```
# 1. Check current status
./status_check.sh

# 2. If components are down, start them
./start_all.sh

# 3. Verify they started
./status_check.sh

# 4. Should now show "Full pipeline operational (4/4 components)"
```

Note: This is a **diagnostic tool only** - it doesn't start, stop, or modify anything. Use it frequently to monitor pipeline health.

stop_all.sh

Purpose: Graceful shutdown of all IDS pipeline components in correct order with optional cleanup.

What it does:

- Stops ML consumer process
- Stops Suricata → Kafka bridge (if running)
- Stops Suricata IDS daemon
- Optionally stops Kafka and Zookeeper (interactive prompt)

- Optionally unbinds DPDK interfaces and restores kernel drivers (interactive prompt)
- Shows final status of all components

When to use:

- **System shutdown:** Before rebooting or powering off
- **Maintenance:** When making configuration changes
- **Troubleshooting:** Clean slate for restarting components
- **Testing:** Reset pipeline between test runs
- **Resource cleanup:** Free CPU/memory when IDS not needed

How it works (in order):

1. **Stop ML Consumer:** Kills `ml_kafka_consumer.py` process (SIGTERM, then SIGKILL if needed)
2. **Stop Bridge:** Kills `suricata_kafka_bridge.py` process
3. **Stop Suricata:** Kills Suricata daemon, removes PID file
4. **Prompt for Kafka:** Asks user if Kafka should be stopped (default: No)
 - If yes: Runs `kafka-server-stop.sh` and `zookeeper-server-stop.sh`
5. **Prompt for DPDK:** Asks if DPDK interfaces should be unbound (default: No)
 - If yes: Runs `unbind_interface.sh` to restore kernel drivers

Shutdown order explained:

```
ML Consumer (top of chain)
  ↓
Bridge (middle)
  ↓
Suricata (source)
  ↓
Kafka (optional - may be used by other services)
  ↓
DPDK (optional - may be needed for other NICs)
```

PROF

Interactive prompts:

Kafka prompt:

```
Kafka Management
Stop Kafka? (y/N):
```

- **Press 'n' or Enter:** Kafka stays running (useful if other services use it)
- **Press 'y':** Kafka and Zookeeper are stopped

DPDK prompt:

```
DPDK Interface Management
Unbind DPDK interfaces? (y/N):
```

- **Press 'n' or Enter:** Interfaces stay bound to DPDK (for quick restart)
- **Press 'y':** Interfaces restored to kernel drivers (for normal networking)

Force kill handling:

- Tries graceful shutdown (SIGTERM) with 2-3 second wait
- If process still running, uses force kill (SIGKILL -9)
- Ensures clean shutdown even if processes are hung

Final status display:

```
Pipeline Stopped
_____

Status:
  ML Consumer: Stopped
  Suricata:    Stopped
  Kafka:       Running    (if you chose not to stop it)
  DPDK Bound:  1 interface(s) (if you chose not to unbind)
```

Prerequisites:

- **Root/sudo access:** Required for stopping Suricata and DPDK operations
- **Must run as:** `sudo ./stop_all.sh`

Use cases:

Scenario	Kafka?	DPDK?
Quick restart	No (N)	No (N)
Full shutdown	Yes (Y)	Yes (Y)
Change configs	No (N)	No (N)
Restore networking	No (N)	Yes (Y)
Free all resources	Yes (Y)	Yes (Y)

Safety features:

-  **Interactive prompts:** Won't stop Kafka/DPDK without confirmation
-  **Graceful shutdown:** Tries SIGTERM before SIGKILL
-  **Status verification:** Shows what's actually stopped
-  **PID cleanup:** Removes stale PID files

Common workflows:

Full shutdown:

```
sudo ./stop_all.sh  
# Answer 'y' to both Kafka and DPDK prompts
```

Troubleshooting tips:

- **Process won't die:** Script uses force kill after timeout
- **"Permission denied":** Run with sudo
- **Kafka stop fails:** Check if Kafka is at `/opt/kafka`
- **DPDK unbind fails:** Check if `unbind_interface.sh` exists

Note: This script is **safe and reversible** - you can always restart components with `start_all.sh` or individual start scripts.

unbind_interface.sh

Purpose: Reverses DPDK binding - takes interfaces back from DPDK and returns them to Linux kernel for normal networking.

What it does:

- Reads backup configuration to find original driver
- Unbinds interface from DPDK driver (vfio-pci/uiopci_generic/igb_uio)
- Rebinds interface to original kernel driver (e1000e/igb/r8169/bnx2x)
- Brings interface UP and shows status
- Restores normal networking functionality

When to use:

- **After DPDK testing:** Restore interface for normal use
- **Need SSH/networking:** DPDK-bound interfaces can't do normal networking
- **Switching modes:** Moving from DPDK mode to AF_PACKET mode
- **System shutdown:** Return interfaces to normal before reboot
- **Troubleshooting:** Restore interface if DPDK mode has issues

How it works:

1. Checks for root/sudo access (required)
2. Loads configuration from `pipeline.conf`
3. Searches for most recent backup in `logs/interface_backup/backup_*.conf`
4. Extracts original driver and PCI address from backup
5. Runs `dpdk-devbind.py -u <PCI>` to unbind from DPDK
6. Runs `dpdk-devbind.py -b <driver> <PCI>` to bind to kernel driver
7. Brings interface UP with `ip link set up`

8. Displays interface status with `ip addr show`

What gets restored:

Before (DPDK bound):

```
$ ip link show
# Interface missing - controlled by DPDK

$ dpdk-devbind.py --status
0000:02:00.0 'Intel 82599ES' drv=vfio-pci
```

After (Kernel restored):

```
$ ip link show
2: ens33: <BROADCAST,MULTICAST,UP> mtu 1500
    link/ether 00:0c:29:xx:xx:xx

$ dpdk-devbind.py --status
0000:02:00.0 'Intel 82599ES' drv=e1000e
```

Backup file format (what script reads):

```
INTERFACE=ens33
PCI_ADDRESS=0000:02:00.0
ORIGINAL_DRIVER=e1000e
TIMESTAMP=20251025_103045
```

Interactive mode (if backup not found):

```
No PCI address configured, showing all DPDK devices:
[List of bound devices]

Enter PCI address to unbind (e.g., 0000:02:00.0): _

Original driver not found in backup
Common drivers: e1000e (Intel), igb (Intel), r8169 (Realtek), bnx2x
(Broadcom)
Enter original driver name: _
```

Troubleshooting:

- **"dpdk-devbind.py not found"**: Install DPDK or check PATH
- **"No devices to unbind"**: No DPDK-bound interfaces, already unbound

- **"Failed to bind to kernel"**: Check driver module is loaded (`lsmod | grep <driver>`)
- **Interface doesn't appear**: Wait 2-3 seconds, kernel may need time to detect
- **"Permission denied"**: Run with `sudo`

Restoration vs rebinding:

- **Unbind (this script)**: DPDK → Kernel (return to normal)
- **Bind (01_bind_interface.sh)**: Kernel → DPDK (enable high performance)

Note: This is the **opposite of 01_bind_interface.sh** - it restores normal networking. Always run this before system shutdown if interfaces are DPDK-bound.

DPDK-Compatible NIC Vendors and Drivers:

1. Intel (Most Popular & Recommended)

Supported Network Cards:

Model	Price Range	Speed	DPDK Support	Best For
Intel i350	~\$50	1 Gbps	✓ Excellent	Budget-friendly, learning DPDK
Intel X520	~\$100	10 Gbps	✓ Excellent	Most popular for DPDK projects
Intel X710	~\$200	10 Gbps	✓ Excellent	Latest generation, production use
Intel 82599ES	~\$150	10 Gbps	✓ Excellent	10-Gigabit SFI/SFP+ connections

Linux Kernel Drivers (original drivers before DPDK binding):

- **e1000e** - Intel 1 Gbps adapters (i350, 82574L, 82577/82579)
- **igb** - Intel Gigabit adapters (I210, I211, I350)
- **ixgbe** - Intel 10 Gbps adapters (82599, X520, X540)
- **i40e** - Intel 40 Gbps adapters (XL710, X710)

DPDK Userspace Drivers (after binding):

- **vfio-pci** (Recommended) - Modern, secure with IOMMU support
- **uio_pci_generic** - Generic, widely compatible
- **igb_uio** - Legacy DPDK driver (requires custom kernel module)

2. Mellanox (High Performance & Enterprise)

Supported Network Cards:

Model	Price Range	Speed	DPDK Support	Best For
ConnectX-3	~\$50-100	10 Gbps	✓ Excellent	Budget high-speed option

Model	Price Range	Speed	DPDK Support	Best For
ConnectX-4	~\$150-250	25/40 Gbps	✓ Excellent	High throughput applications
ConnectX-5	~\$200-400	25/100 Gbps	✓ Excellent	Data center, ultra-high performance
ConnectX-6	~\$400+	100/200 Gbps	✓ Excellent	Cutting-edge performance

Linux Kernel Drivers (original drivers before DPDK binding):

- `mlx4_core` - Mellanox ConnectX-3 and earlier generations
- `mlx5_core` - Mellanox ConnectX-4, ConnectX-5, ConnectX-6 and later

Note: Mellanox NICs support bifurcated driver model - can use kernel driver AND DPDK simultaneously

3. Broadcom (Enterprise-Grade)

Supported Network Cards:

Model	Speed	DPDK Support	Best For
NetXtreme II BCM57810	10 Gbps	✓ Good	Enterprise servers
NetXtreme BCM5719/5720	1 Gbps	✓ Good	Server networking
NetXtreme-E BCM57xxx	10/25 Gbps	✓ Good	High-performance servers

Linux Kernel Drivers (original drivers before DPDK binding):

- `bnx2x` - Broadcom NetXtreme II 10 Gbps adapters
- `tg3` - Broadcom Tigon3 based Gigabit adapters
- `bnxt_en` - Broadcom NetXtreme-C/E adapters

4. Realtek (Limited Support)

Linux Kernel Drivers:

- `r8169` - Realtek 8169/8168/8111 family (consumer-grade)
- **⚠ Note:** Limited DPDK support; most Realtek chips are consumer-grade, not recommended for production DPDK

DPDK Userspace Drivers Explained

When you run `01_bind_interface.sh`, your NIC switches from kernel drivers to these DPDK userspace drivers:

vfio-pci (Recommended)

- **Description:** Modern, secure driver with IOMMU support
- **Advantages:** Best isolation, security, supports SR-IOV
- **Requirements:** Kernel VFIO support, IOMMU enabled in BIOS
- **Load command:** `sudo modprobe vfio-pci`

uio_pci_generic

- **Description:** Generic UIO (Userspace I/O) driver
- **Advantages:** Built into most kernels, good compatibility
- **Disadvantages:** Less secure than vfio-pci
- **Load command:** `sudo modprobe uio_pci_generic`

igb_uio (Legacy)

- **Description:** Legacy DPDK-specific driver
- **Advantages:** Sometimes more compatible with older NICs
- **Disadvantages:** Requires custom compilation, less secure
- **Load command:** `sudo insmod /path/to/dpdk/build/kmod/igb_uio.ko`

Hardware Requirements for DPDK

Component	Requirement	Example
Interface Type	PCI/PCIe only	 <code>lspci</code> shows device
Connection	Direct PCI bus	× USB adapters won't work
PCI Address	Format: <code>0000:01:00.0</code>	Check with <code>lspci</code>
Memory	Hugepages support	2MB or 1GB pages
CPU	IOMMU support (for vfio-pci)	Enable in BIOS

PROF

What Does NOT Work with DPDK

Device Type	Why It Doesn't Work
× USB Network Adapters	No PCI bus access, uses USB subsystem
× WiFi Adapters	Most wireless cards unsupported
× Virtual Interfaces	Software bridges, tunnels, tap/tun devices
× Bluetooth Networking	Not physical PCI devices
× Consumer-grade NICs	Realtek, cheap USB adapters

Quick Compatibility Check

```
# 1. Check your current NICs and their PCI addresses
lspci | grep -i "ethernet\|network"

# Example DPDK-compatible output:
# 01:00.0 Ethernet controller: Intel Corporation 82599ES 10-Gigabit
SFI/SFP+
# 02:00.0 Ethernet controller: Mellanox Technologies ConnectX-5

# Example NOT compatible:
# (USB adapters won't show in lspci at all - they appear in lsusb
instead)

# 2. Check current driver
ethtool -i eth0 | grep driver

# 3. Verify DPDK official compatibility
# Visit: https://core.dpdk.org/supported/
```

Your Current Setup

Interface: `enx00e04c36074c`

Type: USB Ethernet adapter (Realtek r8152 chipset)

Connection: USB, not PCI

DPDK Compatible: × NO

Solution:  **Use AF_PACKET mode** (currently working correctly!)

Installation Steps:

1. **Install PCIe card** in desktop/server
2. **Verify detection:** `lspci | grep Ethernet`
3. **Check PCI address:** Should show format `01:00.0 Ethernet controller: Intel...`
4. **Update config:** Set `NETWORK_INTERFACE` to new interface (e.g., `ens33`)
5. **Bind to DPDK:** Run `./01_bind_interface.sh`
6. **Start Suricata:** Run `./03_start_suricata_dpdk.sh`

Official DPDK Resources

- **Supported NICs List:** <https://core.dpdk.org/supported/>
- **DPDK Documentation:** <https://doc.dpdk.org/>
- **Intel DPDK Guide:**
<https://www.intel.com/content/www/us/en/developer/tools/dpdk/overview.html>